

PROF. DR.-ING. MARK SCHUTERA

FULL STACK HANDWERK

UNFINISHED LECTURE NOTES

Copyright © 2026 Prof. Dr.-Ing. Mark Schutera

PUBLISHED BY UNFINISHED LECTURE NOTES

Combobulated with the help of multiple large language model driven tools. Licensed under the Creative Commons Attribution-NonCommercial 4.0 International License ("CC BY-NC-SA 4.0"). You may not use this file for commercial purposes. If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original. You must obtain explicit permission from the author for uses beyond those permitted by this license. To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/>. Unless required by applicable law or agreed to in writing, distributed material is provided on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the license for details.

These notes are, by their very nature, unfinished, and they improve with every reader. If you spot an error, disagree with a framing, or want to add a source, a question, or a position, open a pull request at https://github.com/Quillstacks/LectureMaterial/tree/main/lecturenotes/notes_missingsemester. Every contribution is welcome.



Last updated, May 2026 — v0.0.2

*“SEINE WORT’ UND WERKE MERKT ICH UND DEN BRAUCH,
UND MIT GEISTESSTÄRKE TU ICH WUNDER AUCH.”*

DER ZAUBERLEHRLING

Contents

<i>Projektbericht</i>	9
<i>Development and Production</i>	11
<i>Version Everything: Git & GitHub</i>	23
<i>Dependencies & Structure</i>	35
<i>Integrate the Model</i>	51
<i>Index</i>	65

Introduction

THIS IS AN ADAPTATION of the MIT Missing Semester course material. And oh boy, we need it here in Germany as well! In the Länd of the "Ingenieure", "Handwerk", and "Lehrlings" - we forgot the importance of practice and mastering our tooling skills.

Original course URL <https://missing.csail.mit.edu/>

AS A MECHANICAL ENGINEER BY TRAINING, I have always had this nagging feeling that something important was missing from my education. The more I transitioned into software engineering roles, the more this feeling intensified. Looking at your curriculum, there is no shortage of advanced concepts and theories to learn. This one fills the gap that many computer science programs overlook: practical proficiency with essential tools and workflows that every software engineer relies on. And truth be told, Professors rely on students having this basic proficiency already acquired, or to figure this stuff out on their own - which is fair in academia. For me, software engineering is also a practical discipline, and ought to be taught as such. I need you to know this stuff, so it is also fair to teach it.

THIS COURSE WILL TEACH YOU everything you need to know to become proficient with the command line, version control systems like Git, text editors, shell scripting, and other essential tools that are the backbone of modern software development - in the end you will be able to develop and deploy your software solutions. This will make you a better software engineer, and it will make your life easier latest when you need to go full ML Lifecycle.

Projektbericht

REAL ARTISTS SHIP. - STEVE JOBS

The Why

A lecture that teaches tools and workflows must also assess tools and workflows. Written exams test recall; a project report tests whether you can actually build and deploy something. The Projektbericht is your proof that you have walked the path from empty virtual machines to a running, deployed application.

PICK ANY BLOCK FROM THE COURSE and build a meaningful extension onto it. The extension should add functionality, improve reliability, or strengthen security in a way that goes beyond what the lecture covered. The choice is yours, but each extension must be unique across the class: submit your proposal for approval on a first come, first served basis with me. Once an extension is taken, no other student may choose the same one. If your proposal strikes home with me, we can also discuss small full stack projects on a microcontroller or other open-source projects that you can contribute to, if you prefer that over building on top of the course material.

SUBMIT A WRITTEN REPORT AS PDF. Include your name, matriculation number, and a link to your snapshot. The report must cover:

- Which block you chose and what it covers in the course.
- What you built on top of it, which tools and techstack you used, and why it adds value.
- Problems you encountered and how you resolved them.
- What works, what remains incomplete, and what would you do next.

EVALUATION considers three dimensions: *depth* of the extension beyond the lecture material, *understanding* demonstrated through explanation and a VM snapshot, and *quality* of the writing.

THE PROJEKTBERICHT replaces a traditional exam. You submit a written report documenting your decisions, your architecture, your modular extension, and the problems you solved along the way.

NUDGE replicate the VM snapshots for Apple Silicon using UTM, so future students on ARM Macs can follow along natively. Build something meaningful, that benefits every cohort after yours.

LOOKING FOR INSPIRATION? Explore what tools modern teams actually use at <https://stackshare.io> or browse the ThoughtWorks Technology Radar at <https://www.thoughtworks.com/radar/tools>. Pick something that excites you; the best reports come from genuine curiosity.

Development and Production

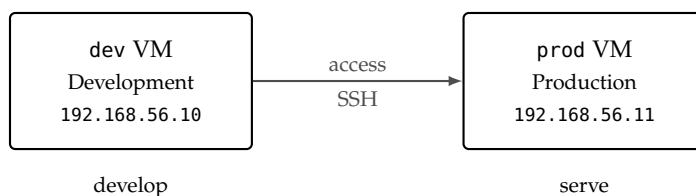
DIVIDE AND CONQUER.

The Why

Most software engineering curricula focus on algorithms, data structures, and the theory of computation. Writing a Python script that runs once on a single laptop is a skill; making that script available as a service, keeping it running, and recovering when it fails is a different one. This course addresses the second set of skills, the tools and workflows that sit between working code and the full stack of setting up and maintaining a running system.

ACROSS TEN LECTURE BLOCKS WE BUILD PIXELWISE, a hand-written digit recognition web application. The starting point is two empty virtual machines; the end point is a deployed service including a frontend, and a backend with a trained model, an API, and a database, all running on Linux servers and connected over a network, with version control, automated deployment, persistent storage, and monitoring. Each tool we introduce is motivated by a specific need that arises in the construction of a fullstack application: Pixelwise.

DEVELOPMENT AND PRODUCTION.



The figure shows the starting arrangement: two virtual machines that play distinct roles throughout the course. The dev VM is the development environment in which code is written, modified, and tested. The prod VM is the production environment in which the deployed application runs. Later blocks add the backend stack, the

PIXELWISE is the narrative thread of the entire course. Users draw a digit on a pixelated 28×28 canvas in the browser, the backend runs an image classifier trained on MNIST, the model predicts class and confidence, and results are stored in a database.

THIS ENABLES YOU to deploy your own products and services in the future.

Figure 1: The two-machine setup. The dev VM is where code is written and tested; the prod VM is where the deployed application runs. Code travels from dev to prod through a deliberate deployment step over SSH.

frontend, persistence, and automation between the two, but the separation between development and production stays intact from this block onward.

Hands On Experience

CONSIDER A SMALL WEB APPLICATION running on your laptop. A classmate two time zones away wants to try it; you send them the address. It works, until you close the lid to catch a train. That evening the Wi-Fi has changed, the address is different, and the process you started in the terminal is gone.

A SERVER is a machine whose job is to be available when no one is watching. It stays powered on, holds a stable address, and answers requests around the clock. Your laptop is where code is written and changed; the server is where code meets users. Keeping these two roles on separate machines is the first design decision of the course.

THIS FIRST BLOCK focuses on machines and environments rather than on code. Before introducing a programming language, a version control system, or a framework, we need a place which is always up, to run programs, isolated from experiments, and a place for development. By the end of the block you will have provisioned both virtual machines,

- set up a server and a development machine as virtual machines on your laptop,
- learned the subset of Linux needed to navigate a server from the command line,
- and configured remote access over SSH using key-based authentication.

COUNT THE HOURS your laptop is actually reachable at a stable address. Subtract the time it is closed, asleep, on battery, behind a café router, or rebooting after an update. What remains is the service level a laptop alone can offer.

TWO MACHINES, ONE MENTAL MODEL. dev at 192.168.56.10 stands in for your laptop. prod at 192.168.56.11 stands in for the always-on host. Later lectures fill in the bridge.

Virtual Machines

A VIRTUAL MACHINE IS A SIMULATED COMPUTER running inside your real computer. The software that creates and runs virtual machines is called a hypervisor. The hypervisor partitions the host's CPU, memory, and disk, and presents each virtual machine with what appears to be its own hardware. From the guest operating system's perspective, it is running on a physical machine.

HYPERVISORS are often categorised as type 1 or type 2. A type 1 hypervisor runs directly on the hardware, with the host operating system built into the hypervisor itself. Examples include VMWare ESXi and Microsoft Hyper-V in its server role; cloud providers use type 1 hypervisors to host the virtual machines you rent. A type 2 hypervisor runs as an application on top of a regular operating system. Oracle VirtualBox Manager is a type 2 hypervisor, which is why we can install it as a normal program on Windows, macOS, or Linux. The conceptual model is the same in both cases; the differences are in performance and deployment context.

ISOLATION is the primary reason to use a virtual machine. A misconfiguration, an uninstalled package, or a destructive command inside the guest does not affect the host operating system. Because the hypervisor abstracts away the hardware, the same VM runs identically on Windows, macOS, and Linux hosts, which is the basis of the reproducibility argument.

THERE ARE COSTS. A virtual machine executes a full operating system on top of the host, which carries a performance overhead that is typically small for I/O-bound work and larger for CPU-bound work. Each VM reserves some amount of RAM, disk space, and CPU time from the host, so the number of concurrent VMs is limited by available resources. Boot time is longer than launching an application but shorter than booting physical hardware. These costs are the price paid for isolation, reproducibility, and rollback.

HOST-ONLY NETWORKING puts dev and prod on a private subnet, typically 192.168.56.0/24, that only the host and the guests can reach. The guests get internet through a second, NAT-mode adapter, so they can install packages without being exposed to the outside world. We assign static IPs, 192.168.56.10 for dev and 192.168.56.11 for prod, so that every command in the course refers to the same addresses.

ORACLE VIRTUALBOX MANAGER at <https://www.oracle.com/virtualization/virtualbox/> is the hypervisor we install on the host. Virt-manager and VMWare serve the same role with different trade-offs, and we could run this course on any of them.

A SNAPSHOT captures the entire machine state at a point in time, including disk contents, memory, and running processes. Snapshots are inexpensive to create and well-suited to exploratory work; making one before a risky change and restoring it afterwards is a standard pattern.

OUTSIDE THE CLASSROOM, a production host is usually a rented or purchased server in a data centre, and the development machine is the laptop or workstation in front of you. Running both as VMs inside a single laptop is a teaching shortcut: it preserves the two-machine separation without asking you to rent hardware, and everything we learn here transfers directly when prod is later replaced by a cloud instance reached over the public internet.

Linux on the Server

LINUX is a family of Unix-like operating systems built around a common kernel, first released by Linus Torvalds in 1991. A distribution combines the kernel with a package manager, system libraries, and user-space tools into a coherent product. Ubuntu, Debian, Fedora, and Arch are distributions that differ in release cadence and defaults but share most of what matters at the command line.

A SERVER IN THIS SECTION is a computer configured to run services and accept connections over a network, typically without a graphical session. You interact with it through a shell, a program that reads commands and prints their output; the shell we use is Bash, the default on Ubuntu.

FIVE COMMANDS are sufficient for the first session. `ls` lists a directory, with `-la` showing permissions, ownership, and hidden files. `cd` changes directory; `cd ..` moves up one level and `cd ~` returns home. `pwd` prints the current path, `mkdir` creates a directory, and `cat` prints a file to the terminal.

LINUX IS A MULTI-USER OPERATING SYSTEM. Every file belongs to a user and a group and carries three permission sets, for owner, group, and everyone else. Each set independently grants read, write, or execute access. `whoami` reports the current user, and `ls -la` shows ownership and permissions:

```
-rw-r--r-- 1 user user 1234 Jan 15 10:00 config.txt
drwxr-xr-x 2 user user 4096 Jan 15 10:00 mydir/
```

TWO COMMANDS modify these. `chmod` changes permissions and `chown` changes ownership:

```
chmod 600 ~/.ssh/id_ed25519          # owner-only read and write
sudo chown user:user file.txt        # assign user and group
```

The principle of least privilege applies throughout the course: grant each user, process, or service only the access it needs, so that narrow defaults limit the blast radius of a compromise or a mistake.

UBUNTU at <https://ubuntu.com/> is the distribution we use on both VMs. Long-term support releases receive five years of security updates, which is why we pick the LTS ISO rather than the latest release.

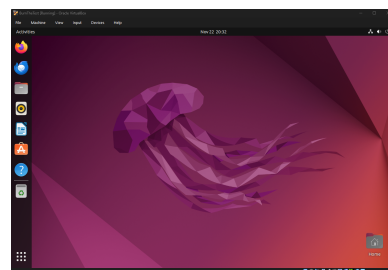


Figure 2: A running Ubuntu guest inside the Oracle VirtualBox Manager. The guest believes it owns a physical machine; the hypervisor is what makes that illusion hold.

THE UNIX PHILOSOPHY shapes the design of these tools. Each program does one thing well, programs pass streams of text to each other, and configuration lives in plain files. The result is a small vocabulary of composable commands that combine into pipelines no single command was designed for.

READING THE MODE STRING from left to right, the first character tells you whether it is a file or directory, then three groups of `rwX` for owner, group, others. A dash means the permission is not granted. The leading `d` marks a directory.

SSH, the Only Door

SSH, THE SECURE SHELL, is a protocol for operating a remote machine from a terminal over an encrypted channel. In the setup used in this course, SSH is the only means of access to the prod VM: there is no graphical login and no console attached. Misconfiguring SSH can lock a user out of the machine, which is why key material and the daemon's configuration are treated carefully in the exercises.

THE BASIC COMMAND takes the form `ssh <username>@<host>`; for example, `ssh produser@192.168.56.11` opens an encrypted connection to the server as the produser account. All input and output travel through this channel until the session is closed with `exit` or `Ctrl+D`.

PASSWORD-BASED AUTHENTICATION is vulnerable to guessing, brute-force attacks, and phishing. SSH supports an alternative in which the user presents a cryptographic key pair. The private key is kept on the client and never transmitted; the public key is placed on the server. During authentication the server issues a challenge that only the holder of the private key can answer, proving identity without the key itself ever leaving the client.

GENERATE AN ED25519 KEY PAIR ON dev:

```
ssh-keygen -t ed25519 -C "you@example.com"
```

The command writes the private key `id_ed25519` and the public key `id_ed25519.pub` to `~/.ssh/`. The private key must have permissions 600, owner-only read and write, or SSH refuses to use it.

COPY THE PUBLIC KEY TO THE SERVER:

```
ssh-copy-id produser@192.168.56.11
```

This appends the key to `~/.ssh/authorized_keys`. Subsequent connections authenticate using the key pair and do not prompt for a password.

ONCE KEY AUTHENTICATION IS VERIFIED, password authentication can be disabled on the server. Edit `/etc/ssh/sshd_config` to set `PasswordAuthentication no`, then restart the daemon:

```
sudo systemctl restart ssh
```

OPENSSH at <https://www.openssh.com/manual.html> is the implementation shipped with Ubuntu and most other Linux distributions. The client on dev and the daemon on prod are both part of this project.

PuTTY at <https://www.putty.org/> is a legacy SSH client for Windows, from the years before OpenSSH shipped with the operating system. Windows 10 and later include `ssh.exe` natively, so PuTTY is rarely necessary; you may still encounter it in older tutorials and on locked-down corporate machines.

VS CODE REMOTE-SSH at <https://code.visualstudio.com/docs/remote/ssh> runs the editor locally while the language server, terminal, and file system live on the remote host. It reads `~/.ssh/config` and reuses the same keys as the `ssh` command, so any host you can reach with `ssh user@host` is one click away inside the editor.

ED25519 keys use elliptic-curve cryptography and produce shorter keys than the older RSA algorithm while providing comparable or stronger security. They are the default choice on modern SSH clients.

MANAGING PRIVATE KEYS. Private keys live in `~/.ssh/` on the machine that initiates connections, protected by filesystem permissions and, when generated with a passphrase, by symmetric encryption. `ssh-agent` holds the decrypted key in memory for the session so the passphrase is typed once per login. Password managers such as KeePass at <https://keepass.info/> or 1Password can store the passphrase and, with their SSH-agent integrations, supply the key to `ssh` without writing it to disk in plaintext. Keys are never synced to cloud storage in the clear, and a separate key pair per machine keeps the blast radius of a compromised laptop local.

After this change, password-based login attempts are rejected before credentials are even evaluated, which eliminates brute-force and credential-stuffing as viable attacks against the machine. The exercise introduces nano as a terminal editor and `systemctl` as the interface to the `systemd` service manager, both of which return throughout the course.

Exercise: Provisioning Virtual Machines

INSTALL THE ORACLE VIRTUALBOX MANAGER for your host OS with default settings, and download two Ubuntu LTS ISOs: the Desktop edition for dev and the Server edition for prod. The asymmetry is deliberate, dev carries a graphical environment so you can run an editor and a browser locally, while prod stays headless because a production server has no business running a desktop. You can also decide to run dev on your machine directly, but this is not further supported in the exercises.

CREATE THE dev VM in the Oracle VirtualBox Manager. Click New and configure:

- Name dev, Type Linux, Version Ubuntu (64-bit)
- Memory at least 4096 MB, CPUs at least 2
- Hard disk at least 25 GB, VDI, dynamically allocated

CREATE THE prod VM with the same procedure but tighter resources, since it runs headless:

- Name prod, Type Linux, Version Ubuntu (64-bit)
- Memory at least 2048 MB, CPUs at least 2
- Hard disk at least 10 GB, VDI, dynamically allocated

ATTACH THE MATCHING ISO to each VM. VirtualBox offers an unattended installation checkbox during creation, don't select it, you want the interactive installer. Once a VM exists, select it in the manager and open Settings, Storage. Under Storage Devices you will see a controller, IDE or SATA, with an Empty optical drive beneath it. Click that Empty entry, then on the right under Attributes click the small disc icon next to Optical Drive, choose Choose a disk file, and pick the Desktop ISO for dev or the Server ISO for prod. Confirm with OK, start the VM, follow the installer, when in doubt go with the defaults and just forward-enter.

AFTER THE FIRST BOOT OF EACH VM (BOTH DEV AND PROD), INSTALL THE OPENSSSH SERVER TO ENABLE REMOTE ACCESS:

```
sudo apt update
sudo apt install openssh-server
```

This step is required for both Desktop and Server editions. Reboot after installation.

EXERCISES are for practice and reinforcing concepts. Work through them yourself first, break things, discuss, this is not a time trial. If your VM ends up in a state you cannot recover, restore the B1-fresh snapshot and start over. At the end of the session, take a snapshot named B1-complete, your safety net for the next block.

WHERE TO GET THEM. Virtual-Box: <https://www.oracle.com/virtualization/virtualbox/>. Ubuntu Desktop LTS ISO: <https://ubuntu.com/download/desktop>. Ubuntu Server LTS ISO: <https://ubuntu.com/download/server>. Pick the LTS release in both cases, not the interim one, so support extends across the semester.

INSTALLER CHOICES that matter: use distinct credentials per VM, user/password on dev and produser/prodpassword on prod. The asymmetry is deliberate, you should feel that logging into prod is a different act from working on dev.

WHY THE SPREAD. Ubuntu Desktop with GNOME wants roughly 4 GB of RAM and 25 GB of disk to feel responsive. Ubuntu Server has no GUI and runs comfortably in a quarter of that. Sized this way the pair fits inside a 6 GB / 35 GB envelope on the host.

HOST-ONLY NETWORK. In File, Host Network Manager (or File, Tools, Network Manager on version 7+), create a host-only network if none exists, typically 192.168.56.1/24. **Disable DHCP** for this network so you can assign static IPs manually.

CONFIGURE THE NETWORK ADAPTERS FOR EACH VM: Select the VM in VirtualBox Manager and open Settings, then go to the Network section. For Adapter 1, check "Enable Network Adapter" and set it to NAT (for internet access). For Adapter 2, select the Adapter 2 tab, check "Enable Network Adapter," set it to Host-only Adapter, and choose the host-only network you created.

ASSIGN STATIC IPs INSIDE EACH VM:

1. Boot the VM and log in.
2. Run `ip a` to list network interfaces. The host-only adapter is usually `enp0s8`, but confirm whether the name shows up in the output.
3. Edit the Netplan config with `sudo nano /etc/netplan/00-installer-config.yaml` Save and exit nano with (`Ctrl+X, Y, Enter`).
4. For dev, set the config as:

```
network:
  version: 2
  ethernets:
    enp0s8:
      addresses:
        - 192.168.56.10/24
```

5. Apply the config with `sudo netplan apply`
6. Check the new address with `ip a` again; you should see the static IP assigned to `enp0s8`.

VERIFY CONNECTIVITY: On dev, run `ping 192.168.56.11` to test if prod responds. If you get no reply, double-check the Netplan configuration (correct interface and address), ensure Adapter 2 is set to Host-only in both VMs, and confirm that both VMs are running and attached to the same host-only network.

```
user@dev:~$ ping 192.168.56.11
PING 192.168.56.11 (192.168.56.11) 56(84) bytes of data.
64 bytes from 192.168.56.11: icmp_seq=1 ttl=64 time=1.63 ms
```

MISSING ADAPTER 2? Power-off the VM, make sure you are in Expert-mode, go to Settings, Network, select Adapter 2, check "Enable Network Adapter," and set it to Host-only Adapter.

OPEN A TERMINAL (DESKTOP). On Ubuntu Desktop press `Ctrl+Alt+T`, or open Activities (top-left) and type "terminal"; press Enter. You can also right-click the desktop and select "Open Terminal" if the menu is present.

For prod, use `192.168.56.11/24` instead.

NETPLAN PERMISSION WARNING. Netplan will warn if a configuration file is writable by group/others or not owned by root. This is a security check to avoid loading untrusted configuration files; expected ownership is `root:root` and modes such as `644` for the YAML files.

TROUBLESHOOTING. If the interface name is not `enp0s8`, use the correct one from `ip a`.

Figure 3: ping

Exercise: Navigating Linux

THIS EXERCISE turns the five commands and the permission model into something your hands remember. You walk the directories common to every Linux system, identify yourself to the machine, and read and change a file's mode string. By the end, the output of `ls -la` should read as a description you can speak aloud rather than a blob of symbols.

EXPLORE THE FILESYSTEM by visiting `/home`, `/etc`, and `/opt` in turn with `cd` and `ls -la`, and read two files with `cat`:

```
cat /etc/hostname
cat /etc/os-release
```

CREATE A WORKSPACE and check that you are where you think you are:

```
mkdir ~/workshop && cd ~/workshop
pwd
touch notes.txt
rm notes.txt
```

`pwd` is a reassurance command. When something does not behave as you expect, asking the shell where it thinks it is usually explains the problem faster than re-reading the error.

UNDERSTAND WHO YOU ARE and look at the permissions on your new file:

```
whoami
id
ls -la notes.txt
```

Decode the first column of `ls -la` against the margin note on mode strings from Block 2. Name aloud what each of the ten characters means; if you can do that without looking, permissions have stopped being mysterious.

CHANGE THE MODE AND WATCH IT:

```
chmod 600 notes.txt
ls -la notes.txt
```

The next exercise will require `600` on your SSH private key, or OpenSSH refuses to use it; you have now seen what `600` looks like and why ownership matters.

TEN-FINGER TYPING is part of the toolchain once the terminal is your main tool. A server has no menus, so your throughput is bounded by how quickly you can speak to the shell without looking at the keys; every hunt for a slash, tilde, or brace is attention spent on the keyboard instead of the problem. Ten minutes a day at <https://www.keybr.com/>, <https://typing.io/>, or <https://monkeytype.com/> is enough to remove that tax over a few weeks.

THE FILESYSTEM AS MAP. `/home` is where users live, `/etc` holds configuration, and `/opt` is where PixelWise will be installed in a later block. These locations reflect decades of convention you will meet on every Linux machine; knowing them lets you locate things by instinct.

A THOUGHT EXPERIMENT ON LEAST PRIVILEGE. Your user account can read `/etc/os-release` but cannot edit `/etc/ssh/sshd_config` without `sudo`. Why does that separation exist, and what would change if the web service we deploy in a later block ran as `root`? Name one file an attacker who compromised such a service could reach that a non-root service could not; the mental habit matters more than the exact file.

Exercise: SSH and Key-Based Access

THIS EXERCISE walks through three states of remote access in order: password login, key-based login, and password login disabled. The point is not the commands but to recognise which state you are in at any moment and why each is stronger than the last.

FIRST, LOG IN WITH A PASSWORD. From dev, open a session to prod:

```
ssh produser@192.168.56.11
```

Type your password, look around briefly, and log out with `exit`. This is the baseline. Every later step is interpretable as an upgrade over this state.

GENERATE A KEY PAIR on dev and install it on prod:

```
ssh-keygen -t ed25519 -C "you@example.com"
ssh-copy-id produser@192.168.56.11
```

Reconnect with `ssh produser@192.168.56.11` and notice the password prompt is gone. Log in and inspect what `ssh-copy-id` wrote for you:

```
cat ~/.ssh/authorized_keys
```

DISABLE PASSWORD AUTHENTICATION. Edit `/etc/ssh/sshd_config` with `sudo nano`, set `PasswordAuthentication no`, and restart the daemon:

```
sudo systemctl restart ssh
sudo systemctl status ssh
```

The status output should show `active (running)`. If it does not, keep your current session open and fix the configuration before logging out; a locked-out VM is rescued by restoring the snapshot, not by wishful thinking. From a machine without the key, the daemon now rejects the attempt without even asking for a password.

TAKE A SNAPSHOT of both VMs once everything works, naming them B1; this is your known good state for the rest of the course. Then deliberately break something inside a VM, watch the fallout, and restore the snapshot. You now have physical evidence that the safety net holds.

WHAT JUST HAPPENED.

`ssh-copy-id` appended the contents of `id_ed25519.pub` on dev to `~/.ssh/authorized_keys` on prod. At the next connection the daemon issued a challenge that only the private key on dev could answer; no secret material ever left the client. Seeing your public key in a readable file anchors the whole exchange to something concrete.

NUKE THE MACHINE. From prod, run `sudo rm -rf /` to delete all files on the machine. Restore from the snapshot.

RUN PROD WITHOUT GUI. In future run prod headless without GUI from the VirtualBox Manager. You can still log in with SSH and run commands, but there is no desktop to interact with. This is how a real production server works.

Self-Reflection and Recap

SELF-REFLECTION Questions which can guide your thoughts during and after the exercises:

- Why do we use two separate machines instead of running everything on one?
- Why is disabling password authentication a stronger protection than picking a longer password?
- What does the principle of least privilege mean in the context of file ownership, and where did it show up in today's workshop?
- If your laptop died tomorrow, how much of today's setup could you reproduce, and what would you lose?
- Which of the five Linux commands do you still have to look up, and which one surprised you?
- How does the two-machine model map to a real cloud deployment with staging and production? How is our development machine different from staging?

RECAP of Key Concepts:

- Virtual machines isolate experiments from the host and make snapshots a first-class safety net.
- The two-machine architecture separates development from production at the hardware level.
- Files on Linux carry owners, groups, and three sets of permissions, and `chmod` and `chown` manage both.
- SSH is the only door into the server, and key-based authentication replaces passwords with cryptography that cannot be guessed.

MILESTONE. So far, we own the machine. The next step is to own the way we build software on it. In the next chapter we introduce Git and GitHub: not just a version history, but the collaboration layer that lets a team branch, review, and merge work in parallel. It is the tool that turns two developers editing the same file from a disaster into a workflow.

A NOTE ON WHAT COMES NEXT. The commands you ran in this session are reproducible but not yet reproduced. You ran them in a terminal, they worked, and they are gone. In the next block we place them under version control and turn the sequence into a script that any future VM can run end to end, without you remembering anything.

WITHOUT SHIPPING ANYTHING YET, we already have two machines that talk to each other, a secure door, and a known good state to fall back on.

TEASER. You and a classmate both want to improve PixelWise at the same time. How do you work on the same codebase without overwriting each other's changes, review what the other person did, and merge the results safely?

Version Everything: Git & GitHub

GIT A LIFE.

The Why

Block 1 gave us two machines and a way to reach one from the other. We can write code on dev and, eventually, deploy it to prod. But nothing yet records what changed between deployments, who changed it, or how to undo a mistake. The moment two people touch the same file, the same function, or one experiment needs to coexist with another, the workflow breaks down entirely.

VERSION CONTROL solves an entire family of problems at once. It records every change with a message, a timestamp, and an author, so the full history of a project is always available. It lets you return to any previous state in seconds, turning risky experiments into cheap, reversible ones.

PARALLEL DEVELOPMENT becomes natural. Multiple people work on the same codebase without overwriting each other's changes. A new feature, a bug fix, and a refactor can all proceed simultaneously on isolated branches and be integrated later with explicit, reviewable merge steps.

DOWNTIME SHRINKS as a direct consequence. Developers merge through a guided review step that catches conflicts and errors before they reach main, deploying only code that has already been integrated and verified. When something still slips through, you can roll back to the last known-good state in a single command instead of debugging under pressure while users wait.

COMBINED WITH A HOSTING PLATFORM like GitHub, version control becomes the backbone of code review, continuous integration, issue tracking, and project management, built around the software development process.

PIXELWISE needs to live on two machines. Right now there is no reliable way to move code from dev to prod and keep both in sync. Git gives us exactly that: a shared history that either machine can pull from, so deployment becomes a deliberate, testable, repeatable step instead of manual file copying.

GitHub: <https://github.com>

Hands On Experience

CONSIDER THIS SCENARIO: you and a classmate are both improving the PixelWise classifier. You rewrite the data loader; your classmate changes the model architecture. You each work on your own laptop for a few hours, then try to combine the results. Without a system for tracking changes, combining means copying files back and forth, comparing line by line, and hoping nothing was lost. Worse still, if both of you would have worked on the same machine, overwriting each others files as you go.

GIT IS THAT SYSTEM. It tracks exactly what changed in every file, when, and by whom. It lets you branch off to try an idea, or build a feature without disturbing stable code, merge branches back together with full visibility into conflicts, and review each other's contributions before they reach the shared codebase. The design principle is simple: never lose work, and never wonder what happened.

THIS SECOND BLOCK introduces Git and GitHub as the version control layer for PixelWise. By the end you will have

- understood the Git data model and why it makes history tamper-proof,
- practised the staging workflow of editing, staging, and committing changes,
- created branches, merged them, and resolved conflicts,
- pushed code to a remote repository on GitHub and collaborated through pull requests,
- and configured SSH-based authentication so Git operations are seamless from both VMs.

EVERY TEAM that ships software uses version control. It is not an optional workflow preference; it is the infrastructure that makes collaborative, iterative development possible at any scale. As projects grows, interface contracts, clear agreements on inputs, outputs, and responsibilities between components, will become increasingly important. Version control lays the foundation by making every change visible and reviewable, so contracts can be enforced rather than assumed.

The Git Data Model

GIT IS NOT A BACKUP TOOL. It is a content-addressable filesystem with a version history layered on top. Understanding the data model makes every command intuitive.

Blob A file's contents, stored by its SHA-1 hash. The name is irrelevant; only the content matters.

Tree A directory listing: maps filenames to blobs (files) or other trees (subdirectories).

Commit A snapshot of the entire project at a point in time. Contains: a pointer to the root tree, the author, a timestamp, a message, and a pointer to the parent commit(s).

Branch A lightweight, movable pointer to a commit. `main` is a branch. Creating a branch is instantaneous; it just creates a new pointer.

HEAD A pointer to the branch you are currently on. When you commit, `HEAD` moves forward.

EVERY COMMIT is identified by a SHA-1 hash, a 40-character hexadecimal string like `a3f2b7c...`. This hash is computed from the commit's contents. If anything changes, even a single character in a single file, the hash changes. This makes Git history tamper-proof.

DATA HYGIENE follows directly from the data model. Because Git stores every version of every blob permanently, committing a large dataset means carrying it in the repository history forever, even if you delete the file later. A 500 MB training set committed once, modified twice, and then removed still occupies 1.5 GB of history. Repositories should contain code, configuration, and small metadata files. Large or frequently changing binary assets, datasets, and model weights do not belong in Git.

The `.gitignore` file, covered in its own subsection below, is the practical mechanism for keeping these artefacts out of the repository.

Git was created by Linus Torvalds in 2005 to manage the Linux kernel source code. It is now the de facto standard for version control in software development.

Git: <https://git-scm.com/>

Pro Git book (free): <https://git-scm.com/book/en/v2>

DEDICATED TOOLS fill the gap. Git LFS replaces large files with lightweight pointers. Hugging Face Hub hosts datasets and model weights with built-in Git LFS. DVC tracks data pipelines and links datasets to experiments. Weights & Biases versions datasets alongside experiment logs.

Git LFS: <https://git-lfs.com>

Hugging Face Hub: <https://huggingface.co>

DVC: <https://dvc.org>

W&B: <https://wandb.ai>

The Staging Workflow

GIT HAS THREE AREAS:

- *Working directory* The files on disk. What you edit.
- *Staging area* What you have marked for the next commit. A deliberate selection.
- *Repository* The committed history. Permanent, immutable snapshots.

THE CORE COMMANDS:

```
git status          # what changed? what is staged?
git add file.py     # stage a file for the next commit
git add .           # stage everything (use with care)
git commit -m "message" # commit the staged changes
git diff            # unstaged changes vs last commit
git diff --staged   # staged changes vs last commit
git log             # show commit history
git log --oneline   # compact view: one line per commit
```

Branching and Merging

BRANCHES LET YOU WORK ON IDEAS IN ISOLATION. The main branch is the stable baseline. Feature branches diverge, evolve, and merge back when ready.

```
git branch          # list branches
git branch feature-x # create a new branch
git checkout feature-x # switch to it
git checkout -b feature-y # create and switch in one step
git branch -d feature-x # delete after merge
```

WHEN THE FEATURE IS READY, merge it back and clean up:

```
git checkout main
git merge feature-x
git branch -d feature-x
```

THE NAPKIN CYCLE as easy as it gets:

```
git pull
git add <files>
git commit -m "..."
```

git push

Pull the latest changes, stage your work, commit it, push it. Every other command is a variation on this loop.

Write commit messages in the imperative mood: "Add classifier module" not "Added classifier module." The message completes the sentence "This commit will ...". Keep the first line under 72 characters. For a gallery of how not to do it, visit <https://whatthecommit.com>.

WHEN TO COMMIT? Commit when you have completed one logical change: a bug fix, a new function, a config update. If you struggle to write a short commit message, you probably changed too many things at once. If you go hours without committing, you are accumulating risk. Small, frequent commits make history useful; large, rare commits make it a wall of text.

```
(\ /)
(0.o)
(> <) Bunny approves these changes.
```

Delete branches after merging. Stale branches accumulate fast and make git branch unreadable. If the branch is merged, it is safe to delete; the history lives on in main.

Merge Conflicts

A MERGE CONFLICT occurs when two branches changed the same line in the same file. Git cannot decide which version to keep, so you must resolve it manually.

GIT MARKS THE CONFLICT in the file. The block between «<<<<< HEAD and ===== is what the current branch has; HEAD always means “the commit you are on right now.” The block between ===== and >>>>> feature-x is what the incoming branch has.

```
<<<<<<< HEAD
result = model.predict(x)
=====
result = classifier.classify(x)
>>>>>>> feature-x
```

Edit the file to keep the correct version and remove the markers. Then stage the resolved file with `git add file.py` and commit with `git commit -m "Resolve merge conflict in file.py"`.

Tags

A TAG MARKS A POINT IN HISTORY THAT MATTERS: a release, a milestone, a known-good state.

```
git tag v0.1                # lightweight tag
git tag -a v0.1 -m "initial setup" # annotated tag (preferred)
git push --tags             # push tags to remote
```

SEMANTIC VERSIONING gives tag names a shared meaning. A version number follows the pattern MAJOR.MINOR.PATCH. Increment PATCH for bug fixes that do not change behaviour, MINOR for new features that remain backwards-compatible, and MAJOR when you introduce breaking changes. A jump from v1.2.3 to v1.3.0 tells users that something was added but nothing they depend on was removed. A jump to v2.0.0 warns them to check for incompatibilities.

Lightweight tags are just a name pointing to a commit. Annotated tags store the tagger, date, and message; use them for releases. We tag v0.1 today; the tag becomes meaningful when you look back at it in Block 8.

tig is a terminal-based Git browser that makes it easy to navigate commits, branches, and tags on a headless Ubuntu machine. Install with `sudo apt install tig`.
<https://jonas.github.io/tig/>
 Semantic Versioning: <https://semver.org>

Remotes and GitHub

```
git clone <url>                # copy a remote repo locally
# fork: copy repo to your GitHub account (GitHub UI, not a git command)
git remote add origin git@github.com:user/pixelwise.git
```

A REMOTE is a copy of your repository on another machine, usually GitHub.

CLONE AND FORK are two ways to start from an existing repository.

A fork is a GitHub concept, not a Git command. It creates a full copy of someone else's repository under your own GitHub account. You clone your fork locally, work on it freely, and propose changes back to the original via a pull request. Forking is the standard workflow for contributing to open-source projects where you do not have write access to the original repository.

`git clone` copies a remote repository to your local machine, keeping the original as the origin remote. You can pull updates and, if you have permission, push changes back.

Pull Requests

PUSH sends your local commits to the remote. Until you push, your work exists only on your machine.

FETCH does the opposite: it downloads new commits from the remote but leaves your working directory untouched, so you can inspect what changed before merging.

PULL combines fetch and merge in one step, updating your local branch to include the remote's latest commits.

```
git push -u origin main    # push and set upstream
git pull                  # fetch + merge from remote
git fetch                  # fetch without merging
```

THE `-u` FLAG links your local branch to the remote branch. After running `git push -u origin main` once, Git remembers the connection and future `git push` and `git pull` work without specifying the remote or branch.

A PULL REQUEST is a proposal to merge a branch into another branch, typically a feature branch into `main`. It is the standard mechanism for code review: your changes are visible, reviewable, and discussable before they become part of the main codebase.

GitHub is not Git. Git is the version control system; GitHub is a hosting platform that adds pull requests, issues, CI/CD, and collaboration features on top of Git. Alternatives include GitLab and Bitbucket.

GitHub: <https://github.com/>
 GitHub Docs: <https://docs.github.com/>

PULL REQUESTS AND CI. Because a PR represents a well-defined set of changes that has not yet reached `main`, it is the ideal moment to run a test suite, a linter, or any other check automatically. If the tests fail, the merge is blocked and `main` stays healthy. We will wire up this automation in a later block; for now, recognise that the PR is where human review and machine verification meet.

The .gitignore File

NOT EVERYTHING BELONGS IN VERSION CONTROL. The .gitignore file tells Git which files and directories to exclude from tracking:

```
# Python
__pycache__/*
*.pyc
.venv/

# Secrets
.env

# Data and models (too large, reproducible)
data/
models/*.pkl

# Editor files
.vscode/
*.swp
```

THIS INCLUDES ALL CREDENTIALS: API keys, database passwords, webhook secrets, tokens for AI services. These live on the server in environment variables or protected configuration files, never in the repository. A leaked production database password or API key can be exploited within minutes by automated scanners.

Git history is permanent. A secret committed once is leaked forever, even if you delete the file in a later commit. The old commit still contains it. .gitignore is your first line of defence.

Exercise: Setting Up Git and the Repository

CREATE A GITHUB ACCOUNT if you do not have one. Then open a terminal on the dev VM.

INSTALL GIT. Ubuntu does not ship with Git by default, so install it through the package manager. It does not matter which directory you run this in, apt installs system-wide:

```
sudo apt update
sudo apt install -y git
git --version           # confirm the install succeeded
```

CONFIGURE YOUR GIT IDENTITY:

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

GENERATE AN SSH KEY. SSH keys authenticate you to GitHub from the command line, replacing the password prompt on every push and pull. Before generating, check whether a key already exists so you do not overwrite one you rely on for another server:

```
ls ~/.ssh/
```

If you see a file called `id_ed25519`, that path is already in use. Either reuse it for GitHub as well, or generate a new key under a distinct filename so both keep working. Otherwise, generate the default key pair on the dev VM:

```
ssh-keygen -t ed25519 -C "your@email.com"
```

The tool prompts for a file location. If `~/.ssh/id_ed25519` does not yet exist, press Enter to accept the default; if it does, type a distinct path such as `~/.ssh/id_ed25519_github` so the existing key is preserved. It then prompts for a passphrase, optional but recommended, it encrypts the private key on disk so a stolen key file is not immediately usable.

ADD THE PUBLIC KEY TO GITHUB. Print it to the terminal and select the entire output, beginning with `ssh-ed25519` and ending with the comment:

```
cat ~/.ssh/id_ed25519.pub
```

AT THE END OF THE SESSIONS, take a snapshot named B2-complete, your safety net for the next block.

OPENING A TERMINAL ON UBUNTU. The fastest way is the keyboard shortcut `Ctrl+Alt+T`, which launches the default terminal emulator. Alternatives: press the Super key (the Windows key) and type "terminal", or right-click on the desktop and choose "Open Terminal". Inside a terminal you can open another tab with `Ctrl+Shift+T` and a fresh window with `Ctrl+Alt+T` again.

`sudo` runs the command with administrator privileges, which apt needs to write to system directories. You will be asked for your password the first time. `apt update` refreshes the package index so you get the current version; `apt install -y` installs without prompting for confirmation.

RINGS A BELL? You generated an `id_ed25519` key in Block 1 to log in to the prod VM without a password. That same key is sitting in `~/.ssh/` now, which is exactly why `ls` matters before the next step: accepting the default path would overwrite the production key and lock you out of the server.

MULTIPLE SSH KEYS? If you keep one key for production servers and another for GitHub, tell SSH which to use for each host with an `~/.ssh/config` entry:

```
Host github.com
  HostName github.com
  User git
  IdentityFile
~/.ssh/id_ed25519_github
  IdentitiesOnly yes
```

Then `chmod 600 ~/.ssh/config`. Without `IdentitiesOnly`, SSH may offer the wrong key first and GitHub will reject the connection after a handful of auth failures.

WHAT JUST HAPPENED? `ssh-keygen` created two files in `~/.ssh/`. `id_ed25519` is the private key: never share it, never commit it, never copy it to another machine without good reason. `id_ed25519.pub` is the public key, safe to share, this is the file you paste into GitHub. The `-t ed25519` flag picks a modern signature algorithm:

In GitHub, navigate to Settings, then SSH and GPG keys, then New SSH key. Give it a recognisable title like `pixelwise-dev-vm` so you know which machine it belongs to, paste the public key into the Key field, and save.

TEST THE CONNECTION:

```
ssh -T git@github.com
```

The first time, SSH asks whether to trust GitHub's host fingerprint. Type yes. If the key is set up correctly, GitHub responds with `Hi <username>!` You've successfully authenticated, but GitHub does not provide shell access. The message about shell access sounds like an error but is the intended response: GitHub only allows Git operations over SSH, not interactive shells.

INITIALISE THE PIXELWISE REPO on the dev VM. Pick a stable working directory, your home folder is the natural place, so the project does not get lost in `/tmp` or buried under Downloads:

```
cd ~ # start from your home directory
mkdir pixelwise && cd pixelwise
pwd # confirm: /home/<user>/pixelwise
git init
```

`cd ~` jumps to your home directory from anywhere. `pwd` prints the current path so you can confirm where you are before running `git init`. Avoid initialising a repo inside `/tmp`, on a mounted USB drive, or inside another existing Git repository.

Write a `README.md` with a one-line project description.

Exercise: Staging, Committing, and .gitignore

WRITE A `.gitignore`. Create a `.gitignore` file that excludes Python caches, virtual environments, secret files, dataset directories, model checkpoints, and editor files. Verify it works: create a file called `scratch.pyc`, run `git status`, and confirm Git does not list it. Then remove the test file.

MAKE YOUR FIRST COMMIT. Stage and commit `README.md` and `.gitignore`:

```
git add README.md .gitignore
git commit -m "Add project README and .gitignore"
```

Run `git log` to see your first commit. Run `git status` to confirm the working directory is clean.

THE HABIT TO BUILD: after every commit, run `git status` and `git log --oneline` to confirm what just happened. It takes two seconds and prevents surprises.

Exercise: Branching, Merging, and Conflicts

CREATE A FEATURE BRANCH and make a change to README.md:

```
git checkout -b feature-readme
nano README.md          # add a project description line
git add README.md
git commit -m "Add project description to README"
```

SWITCH BACK TO main and edit the same line differently:

```
git checkout main
nano README.md          # edit the same line, but differently
git add README.md
git commit -m "Add alternative description to README"
```

MERGE AND RESOLVE THE CONFLICT. Now merge the feature branch and observe the conflict:

```
git merge feature-readme
```

Git reports a conflict and pauses the merge. Open README.md with nano README.md. You will see a block that looks like this:

```
<<<<<< HEAD
... your text on main
=====
... text from feature-readme
>>>>>> feature-readme
```

HEAD marks the version sitting on the branch you are currently on, in this case main. The lines after ===== are what feature-readme is bringing in. Choosing the correct version means deciding what the file should look like once both changes are reconciled. You have three options: keep only the HEAD side, keep only the incoming side, or write a merged version that combines parts of both. Then delete all three marker lines, <<<<< HEAD, =====, and >>>>> feature-readme, so the file contains only the resolved content. Save with Ctrl+O and exit with Ctrl+X, then complete the merge:

```
git add README.md
git commit -m "Resolve merge conflict in README"
```

Verify with git log --oneline --graph that both branches are now integrated.

EDITING FILES IN THE TERMINAL.

nano README.md opens the file in a simple terminal editor. The bar at the bottom of the screen lists the shortcuts, where ^ stands for the Ctrl key: Ctrl+O writes the file to disk, Ctrl+X exits, and Ctrl+G shows full help. Vim and VS Code over Remote SSH work just as well if you prefer them.

main OR master? Older Git versions called the default branch master. Since 2020, the convention has shifted to main, and GitHub creates new repositories with main by default. If git checkout main reports that the branch does not exist, run git branch to see what your repo actually calls it. To bring an existing repository in line with the new convention, rename in place with git branch -m master main, then set the default for future git init runs with git config --global init.defaultBranch main.

git status DURING A MERGE. While the merge is paused, git status reports "You have unmerged paths" and lists every file with conflict markers still in it. Run it any time to remind yourself which files still need to be resolved before you can finish the merge.

Exercise: Tags and History Browsing

TAG THE RELEASE. Mark this point in history:

```
git tag -a v0.1 -m "initial server setup"
```

Verify the tag exists with `git tag -l` and inspect it with `git show v0.1`.

INSTALL tig, a terminal-based Git browser:

```
sudo apt install -y tig
tig
```

Navigate the commit history and find your tag. Press `q` to quit.

Exercise: Remotes, Push, and Clone

CONNECT TO GITHUB. Create a repository on GitHub, add it as a remote, push the commits, then push the tag you created in the previous exercise:

```
git remote add origin git@github.com:user/pixelwise.git
git push -u origin main
git push --tags
```

Verify on GitHub that your commits, `.gitignore`, and the `v0.1` tag under the “Releases” tab are all visible.

CLONE TO THE SERVER. SSH into the server and clone PixelWise to its permanent home:

```
sudo mkdir -p /opt/pixelwise
sudo chown produser:produser /opt/pixelwise
git clone https://github.com/user/pixelwise.git /opt/pixelwise
```

PixelWise now lives on both machines. Run `git log --oneline` on the server to confirm the full history arrived.

WHY NO `git push` YET? Tags live locally until you push them. We have not configured a remote yet, that happens in the next exercise. Once the remote exists, `git push --tags` ships every local tag to GitHub, where it appears under the repository’s “Releases” tab.

VS CODE can also browse Git history visually. Open the PixelWise folder via `code .` or Remote SSH, then use the Source Control panel and the GitLens extension to explore commits, branches, and tags with a graphical interface.

REPLACE `user` WITH YOUR GITHUB USERNAME. The `user` placeholder in the remote URL is just a stand-in. Substitute your own GitHub handle, the one shown in your profile URL at `github.com/<your-handle>`, so the URL points at the repository under your account. The same goes for the `git clone` URL on the server below. Pushing to a `user/pixelwise` that you do not own will fail.

SSH INTO `prod` AGAIN. From the `dev` VM, open a session to `prod` the same way you did in Block 1:

```
ssh produser@192.168.56.11
```

The key-based login from Block 1 is still in place, no password should be required. Once you are on `prod`, the prompt changes to `produser@prod`, and the commands below run on the server. Type `exit` when you are done to return to `dev`.

WHY HTTPS ON `prod`, SSH ON `dev`? `dev` pushes commits back to GitHub, so it needs the SSH key you registered earlier. `prod` only ever pulls, it is a deployment target, never an authoring machine. HTTPS clone of a public repository requires no key, no token, no extra setup, which matches how real deployment servers usually have read-only access to the source repository. If your repository is private or you later need to push from `prod`, generate a second SSH key on `prod` and register it on GitHub the same way you did on `dev`.

Self-Reflection and Recap

REFLECTION QUESTIONS:

- Why does changing a single character in a file produce a completely different commit hash?
- What is the difference between `git add` and `git commit`, and why does Git separate the two steps?
- When would you use a branch instead of committing directly to `main`, and what happens when a merge produces a conflict?
- Why should you never commit `.env` files or API keys, and what is the role of pull requests in catching mistakes before they reach `main`?
- Why does PixelWise use SSH keys rather than passwords for Git operations across both VMs?

MILESTONE: Your project lives on GitHub with a README, a `.gitignore`, and a tagged release. The server has the repo cloned at `/opt/pixelwise`. Both machines share the same history. In the next block we turn the manual package installs into a reproducible setup script and tackle dependency management.

Dependencies & Structure

WORKS ON MY MACHINE.

The Why

Block 2 left us with a PixelWise repository on GitHub and a server that has Git and Python installed. Both machines share the same history, both can pull from origin, and the workflow that moves code between them is in place. What is still missing is the layer beneath the code: the system packages, the language runtime, the libraries, and the secrets your application reads (installed or provisioned by `setup-server.sh` and recorded in `.env/.env.example`).

KALTSTART CAPABILITIES, Your colleague clones the repository, runs the code, and immediately hits an Error. You installed a package with a specific version months ago and forgot. They did not. The code is correct; the environment is wrong. The remainder of the day is spent debugging an artefact of the install history rather than contributing real value to the program itself. At the end of this third block we want any fresh VM to go from empty to ready to code with a small, fixed sequence of commands. The two halves of that sequence are the load-bearing idea of the chapter: system packages on one side, Python packages on the other, version controlled in different files.

PIXELWISE now lives on GitHub and on both machines, but the repository contains only `README.md` and `.gitignore`. Before we can write application code, we need a reproducible way to install everything the project depends on, so that dev and prod stay in sync without human memory. Use `setup-server.sh` to provision system packages and the runtime, and keep secrets and runtime configuration in `.env.example` (copied to `.env`).

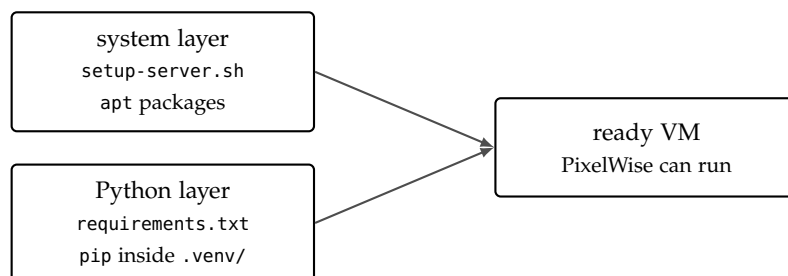


Figure 4: Two layers, two files. `setup-server.sh` provisions the operating system; `requirements.txt` provisions Python inside an isolated virtual environment. Together they reconstruct the environment from scratch on any fresh VM. System packages, the things you install with `apt`, are captured in a shell script called `setup-server.sh`. Python packages, the libraries your application imports, are captured in `requirements.txt` and installed inside an isolated virtual environment. The two files are committed to Git, run on either machine, and produce identical environments without any further intervention.

Hands On Experience

CONSIDER THIS SCENARIO: you hand your PixelWise repository to a classmate at the end of the day. They clone it, type `python3 train.py` without running the provisioning script `setup-server.sh`, and immediately hit `ModuleNotFoundError: No module named 'sklearn'`. You tell them to install scikit-learn. They do, but they get version 2.0 while you wrote the code against 1.4. The training script crashes with a different error. By the time the original problem is fixed, a few hours are gone and neither of you has touched the actual code nor added any value.

REPRODUCIBLE PROJECTS do not require remembering, asking, or guessing. The exact set of packages, the exact versions, the exact configuration contract, are written down once, committed to Git, and replayed on any machine that has the script and the file.

THIS THIRD BLOCK introduces dependency management and project structure for PixelWise. By the end you will have

- captured system-level setup in a reproducible shell script,
- created and activated a Python virtual environment,
- installed packages with pip and pinned them in `requirements.txt`,
- separated secrets from code using `.env` and `.env.example`,
- scaffolded the PixelWise project directory,
- and audited your dependencies for known security vulnerabilities.

THE TWO-COMMAND PROMISE. A well-managed project lets any new developer go from clone to running code with two commands: create the virtual environment and install from a pinned requirements file. No guessing, no mails or messages asking “which version of numpy do I need?” This block is what makes that promise hold.

OUTSIDE THE CLASSROOM, the same files are how cloud deployment works in practice. A continuous-integration runner clones the repository, executes the setup script (for example, `setup-server.sh`), installs from `requirements.txt`, and runs the tests. The reproducibility you build for two VMs scales without modification to a hosted server, a colleague’s laptop, or a CI runner you have never seen.

The Setup Script

PROVISIONING A SERVER is the act of turning a freshly installed operating system into one that can run your application. So far it has been done by typing `sudo apt install` commands at the prompt and reading their output. That works for one-time setups, it does not scale. Software is meant to be scaled.

THE SETUP SCRIPT is a plain Bash file that captures every `apt install` the project needs. It lives in the repository alongside the code and is run on any fresh VM to bring it up to speed:

```
#!/bin/bash
# setup-server.sh: provision a fresh server VM
set -euo pipefail

sudo apt update
sudo apt install -y git python3 python3-pip \
    python3-venv curl
```

THE FIRST LINE OF THE BODY, `set -euo pipefail`, makes the script fail loudly. `-e` aborts on the first error, `-u` aborts on an undefined variable, and `-o pipefail` propagates a failed command in the middle of a pipeline rather than letting a successful command later on mask it. Half-configured machines are a common cause of mysterious bugs; the four-character preamble eliminates most of them.

MARK IT EXECUTABLE and run it:

```
chmod +x setup-server.sh
./setup-server.sh
```

SEE HOW VERSION CONTROL ALREADY PAYS OFF. The script is committed to Git, so when the VM is rebuilt or a new team member joins, `bash setup-server.sh` reproduces the exact same environment. The commands you typed once are written down where the next person can find them.

CONFIGURATION MANAGEMENT is a whole discipline that grew from this small itch. Ansible, Salt, and Puppet declare the desired state of a machine in a structured format and reconcile the live system to match. `setup-server.sh` is the entry-level version: a script you can read end to end. The principle is the same.

Ansible: <https://docs.ansible.com/>

THE SHEBANG LINE `#!/bin/bash` tells the kernel which interpreter to use when you run the file directly. Without it, the file is read as a sequence of commands by whichever shell happens to invoke it, which on Ubuntu is usually `dash`, not `Bash`, and the two differ in subtle ways that surface only when something breaks.

IDEMPOTENCE is the property that running a script twice yields the same result as running it once. `apt install` is idempotent by default: an already-installed package is a no-op. So re-running is always safe, and the script can be used to fix a machine that has drifted.

Virtual Environments

A VIRTUAL ENVIRONMENT is an isolated Python installation. Each project gets its own set of packages, independent of every other project and of the system Python. The mechanism is local: a directory, by convention called `.venv/`, that holds an interpreter, a copy of `pip`, and every library the project depends on. Activating the environment rewrites your shell's `PATH` so that `python` and `pip` resolve to the local copies.

```
python3 -m venv .venv
source .venv/bin/activate
```

THE DIRECTORY IS LARGE and machine specific, often 50 to 200 MB. Thus it is listed in `.gitignore`: never commit it. A teammate clones the repository, creates their own `.venv`, installs from `requirements.txt`, and ends up with the same packages without ever copying the binary contents.

VERIFY THE SWITCH. After activation, which `python` should report a path inside `.venv/bin/`, and `pip list` should show only `pip` and `setuptools`:

```
which python
# /home/user/pixelwise/.venv/bin/python

pip list
# Package      Version
# -----
# pip          24.0
# setuptools 69.5.1
```

A fresh environment starts empty. Every package you install from this point is tracked and intentional. `deactivate` returns the shell to the system Python.

THE RULE IS SIMPLE: every project gets its own `.venv`. Activate it before you work; deactivate when you are done. The one-time cost is a directory and a command; the ongoing benefit is never debugging a version conflict between two unrelated projects on the same laptop.

WHY ISOLATION? Project A needs `numpy==1.24`; project B needs `numpy==2.0`. Without per-project environments, one of them breaks the moment the other is installed. With virtual environments, both coexist on the same machine and make it easy for you to switch between projects seamlessly.

THE DATA SCIENCE ECOSYSTEM often uses `conda` instead of `venv`. `Conda` manages both Python packages and system-level libraries like `CUDA` or `MKL`, which makes it popular for GPU workflows. For web projects, `venv` is the lighter and more standard choice.
Conda: <https://docs.conda.io/>

Managing Packages with pip

PIP IS THE PACKAGE INSTALLER for Python. It downloads packages from PyPI, the Python Package Index at <https://pypi.org/>, and installs them into the active environment. The core cycle has three commands:

```
pip install scikit-learn joblib python-dotenv
pip freeze > requirements.txt
pip install -r requirements.txt
```

`pip install` downloads each named package and its dependencies and places them in `.venv/`. `pip freeze` prints every installed package with its exact version, including transitive dependencies you never asked for directly. `pip install -r` reads such a file and installs everything in it, the step that turns a captured environment back into a live one.

READ THESE THREE LINES AS A CONTRACT. The first installs into the environment. The second writes a snapshot of it to a text file. The third reconstructs the same environment from the file on any machine. Together they are the reproducibility layer for your programming environment.

Anatomy of requirements.txt

ONE PACKAGE PER LINE, pinned to an exact version:

```
scikit-learn==1.4.0
joblib==1.3.2
python-dotenv==1.0.1
```

PINNING MATTERS because packages are a moving target. A package you install today as `numpy>=1.24` may resolve to 1.27 tomorrow, with new features added, new behaviour, new bugs, and possibly new security advisories. Packages are software, and they are subject to change; just the same as your own project. Pinning to `==1.24.3` freezes the resolution and makes switching to newer versions a deliberate act.

VERSION SPECIFIERS. `==` pins exactly, `>=` sets a floor, `~=` allows patch updates: `~=1.4.0` means `>=1.4.0, <1.5.0`. For maximum reproducibility, pin everything with `==`. Looser specifiers are appropriate for libraries that other projects depend on, which is not where most of your work lives. Semantic versioning: <https://semver.org>

COUNT WHAT `pip freeze` WRITES. Three direct installs typically produce twenty or more lines, the rest are transitive dependencies pulled in automatically. The flatness is the point: every package that has to be present for the project to work is listed, even the ones you never typed. The cost is that the file is hard to read; the benefit is that it captures exactly what is on disk.

SECURITY AUDITING. Your dependencies have their own dependencies, and any of them can harbour known vulnerabilities. `pip-audit` checks every installed package against the CVE database: `pip install pip-audit` && `pip-audit`. Run it after every `pip freeze`. In Block 8 we wire it into a cron job so vulnerabilities surface without human labor. `pip-audit`: <https://github.com/pypa/pip-audit> Dependabot: <https://docs.github.com/en/code-security/dependabot>

Configuration: .env and python-dotenv

SEPARATING CONFIGURATION FROM CODE is a foundational principle. Configuration is anything that varies between development, staging, and production: API keys, database URLs, feature flags, debug settings. Code is the part that does not change between deployments.

A `.env` FILE holds key-value pairs that your application reads at startup:

```
SECRET_API_KEY=my-actual-secret-key-here
DEBUG=true
```

In Python, `python-dotenv` loads the file into process environment variables, after which the standard `os.getenv` reads them:

```
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("SECRET_API_KEY")
debug = os.getenv("DEBUG", "false").lower() == "true"
```

The .env.example Pattern

THE REAL `.env` contains real secrets, so it must never be committed. `.env.example` is the contract: it lists which variables exist, explains what each one is for, and provides a safe placeholder. The example file is committed and public. The filled contracts live on the respective machines and stays out of Git.

```
# API key callers must send in the X-API-Key header
# (a secret, never a real value here)
SECRET_API_KEY=replace-me

# Debug mode: when true, FastAPI shows /docs and full
# error tracebacks. Never true in production.
DEBUG=true
```

ON FIRST SETUP CREATE YOUR ACTUAL `.env` by copying `.env.example` and replacing each placeholder with a real value. The contrast is immediate: one is the template, the other is a vault for your secrets.

The Twelve-Factor App: <https://12factor.net/>
 python-dotenv: <https://github.com/theskumar/python-dotenv>

A FILE FOR CONVENIENCE. Environment variables can be exported manually, set inline before a command, or supplied by the `systemd` service file we install in Block 5. A `.env` file is the development-time convenience that mirrors the production-time mechanism: same names, same shape, different source.

The pattern grows block by block. Every new service we add, the API key in Block 5, the database URL in Block 6, the model path in Block 4, lands in `.env.example` as a new line with a comment explaining its purpose. A teammate cloning the repository six weeks from now sees the full surface of the application's configuration in one file.

Optional: When requirements.txt Starts to Hurt

`pip freeze > requirements.txt` captures everything: your direct dependencies and all their transitive dependencies, flattened into one list. Six months later you want to upgrade one package: which of those forty-seven packages did you deliberately install, and which were pulled into that list automatically? Without that distinction, every upgrade is a guessing game. Solving dependency conflicts becomes a nightmare and quickly feels like brute-force.

`pyproject.toml` solves this by separating “what you depend on”, your direct deps loosely pinned, from “what gets installed”, a generated lock file with the full resolved graph. Upgrades become surgical: bump one line, regenerate the lock, done.

```
[project]
name = "pixelwise"
version = "0.1.0"
dependencies = [
    "scikit-learn>=1.4",
    "joblib>=1.3",
    "python-dotenv>=1.0",
]
```

COMPARE this to `requirements.txt`: the project file lists only what you chose to install, with flexible version bounds. The tool resolves the full dependency graph and writes a lock file that pins every transitive dependency exactly. You commit both files; the lock file is your reproducibility guarantee and the project file is your editable surface.

```
# to upgrade a single package and update lock:
poetry update <package>
```

TOOLS LIKE `uv` and `poetry` use the project-file model and are fast becoming the standard. Once the `requirements.txt` workflow is second nature, this is the upgrade that makes dependency management sane at scale.

Python Packaging User Guide: <https://packaging.python.org/en/latest/>
`uv`: <https://docs.astral.sh/uv/>
`Poetry`: <https://python-poetry.org/>

WE STAY WITH `requirements.txt` through the rest of the course because it is the format every Python developer recognises and the only one some hosting environments still accept. The reasoning generalises: when an upgrade is available, learn the workflow beneath it before adopting the tool that abstracts it away.

Optional: The Container Alternative

VIRTUAL ENVIRONMENTS ISOLATE PYTHON PACKAGES. Containers isolate the entire operating system: libraries, system packages, Python version, environment variables, even the filesystem layout. Where `requirements.txt` specifies which Python packages to install, a `Dockerfile` does the whole stack in layers: This Linux image, install these system packages, then install these Python packages.

A MINIMAL `DOCKERFILE` for a Python service looks much like the two scripts you are about to write, compressed into a single declarative file:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "main.py"]
```

The result is a self-contained unit that runs identically on any machine with Docker installed. The trade is opacity: when something fails inside a container, the layers between your code and the kernel multiply, and debugging across them requires familiarity with the layer above and below.

WE DO NOT USE DOCKER IN THIS COURSE, but you should know it exists and why teams adopt it. The vocabulary, image, container, registry, layer, volume, recurs in any later production work; the next-block decisions in this course translate to it directly.

DOCKER and Docker Compose solve the reproducible environment problem at the OS level rather than at the Python level. This course teaches the `virtualenv` path to keep the OS layer visible and learnable. Later Docker enables you to run Linux containers under Windows as well. Docker is the natural next step after this course; it does not replace any of what you learn here, it builds on it. Docker: <https://www.docker.com/> Docker docs: <https://docs.docker.com/get-started/>

WHY WE HOLD OFF. The OS layer is the layer most students are least exposed to. Containers hide it before students have seen it. By the end of this course you will have provisioned, deployed, and monitored a service on bare Ubuntu; lifting that into a container at that point is mechanical, and you will know exactly what each line of the `Dockerfile` means.

Exercise: The Setup Script

CATCHING UP VIA THE REFERENCE TAG. From this block onwards, the GitHub repository is the safety net. The course maintains a public reference repository at <https://github.com/schutera/pixelwise> with a tag at the end of every block. v0.1 marks the end of Block 2; v0.2 will mark the end of this block. If you joined late, missed Block 2, or your repository has drifted from a known good state, reset to the previous tag rather than redoing every prior exercise.

CATCH-UP PROCEDURE. Restore the B1-complete VM image from Block 1, complete the Block 2 Git identity and SSH key setup so the VM has your credentials, then clone the reference repository at the v0.1 tag:

```
cd ~
git clone --branch v0.1 \
    https://github.com/schutera/pixelwise.git
cd pixelwise
git switch -c main
git remote set-url origin \
    git@github.com:<you>/pixelwise.git
```

You are now at the state where Block 2 ended, ready to start Block 3.

BLOCK 2 INTRODUCED THE SETUP SCRIPT AS THE SYSTEM LAYER. On the dev VM, inside the PixelWise repository, create a shell script that captures the system packages installed manually in Block 2:

```
#!/bin/bash
set -euo pipefail

sudo apt update
sudo apt install -y git python3 python3-pip \
    python3-venv curl
```

MARK IT EXECUTABLE AND RUN IT. On a machine that already has the packages, every `apt install` is a no-op; the script reports nothing to do, which is the desired outcome of an idempotent provisioning step:

```
chmod +x setup-server.sh
./setup-server.sh
```

EXERCISES are for practice and reinforcing concepts. Work through them yourself first, break things, discuss; this is not a time trial. Block 2 introduced six topics; the exercises in this block mirror them in the same order, so you can read a concept and immediately practise it.

TAG MAP. v0.1 is the end of Block 2, v0.2 is the end of this block, v0.3 the end of Block 4, and so on. Every tag is a complete, runnable state of PixelWise. Browse them at <https://github.com/schutera/pixelwise/tags>.

TRY IT ON A FRESH VM. The real test of the script is on a machine that has not seen the commands yet. A second prod VM, restored from the B1-complete image, clones the repository, runs `bash setup-server.sh`, and provisions end to end without you typing a single `apt install`.

COMMIT IT. This is the first file that makes provisioning reproducible:

```
git add setup-server.sh
git commit -m "Add server provisioning script"
git push
```

Exercise: The Virtual Environment

BLOCK 2 INTRODUCED VIRTUAL ENVIRONMENTS as the Python isolation layer. On the dev VM, inside the PixelWise repository, create one:

```
cd ~/pixelwise
python3 -m venv .venv
source .venv/bin/activate
```

VERIFY THE SWITCH. which python should print a path inside .venv/bin/. pip list should show only pip and setuptools:

```
which python
pip list
```

A fresh environment is empty by design. The next exercise fills it with intent.

ADD THE ENVIRONMENT TO .gitignore. The .venv/ directory must never be committed. Open the .gitignore from Block 2 and confirm .venv/ is listed; if not, add it now:

```
.venv/
*.pkl
data/
.env
```

Run git status before and after to see how much noise the new entries silence.

NOTICE THE PROMPT CHANGE. After activation, your shell prompt usually gains a (.venv) prefix to remind you which environment is active. If you ever wonder whether you are inside the project's environment, the prompt is the first place to look; which python is the second.

READ THE DIFF. git diff .gitignore shows what you added compared to the Block 2 version. Reading the diff before committing is the habit that catches accidental changes early; running it once now is muscle memory you will reuse every day for the rest of the course.

Exercise: Pinning Dependencies and pip-audit

BLOCK 2 INTRODUCED PIP as the package installer and requirements.txt as the captured environment. With the virtual environment active, install the three direct dependencies PixelWise needs in this block:

```
pip install scikit-learn joblib python-dotenv
```

FREEZE THE ENVIRONMENT into a pinned requirements file:

```
pip freeze > requirements.txt
```

Open requirements.txt and inspect. Note the pinned versions and the transitive dependencies that were pulled in automatically. Count how many lines pip freeze produced compared to the three packages you installed directly; the gap is the dependency graph working invisibly.

AUDIT THE DEPENDENCY GRAPH. Install pip-audit and run it against the active environment:

```
pip install pip-audit
pip-audit
```

Inspect the output. What is a CVE? What would you do if a critical vulnerability were reported in one of your dependencies? The reading you do here pays back in Block 8 when the audit becomes part of the deploy pipeline.

COMMIT THE REQUIREMENTS:

```
git add requirements.txt
git commit -m "Pin core Python dependencies"
```

THE HABIT TO BUILD: after installing or upgrading any package, immediately run `pip freeze > requirements.txt` and commit the change. Stale requirements files are the most common cause of “works on my machine” incidents.

READ ONE CVE. Pick any advisory pip-audit reports, click through to the CVE record, and read the description, the affected versions, and the fixed version. Practising this once now makes the production response straightforward when an alert fires under time pressure.

Exercise: Configuration and Secrets

BLOCK 2 INTRODUCED `.env` AND `.env.example` as the contract for separating configuration from code. Create `.env.example` at the repository root with two entries:

```
# API key (secret, never a real value here)
SECRET_API_KEY=replace-me

# Debug mode (never true in production)
DEBUG=true
```

CREATE YOUR REAL `.env` by copying the example and replacing the placeholder with a value you choose:

```
cp .env.example .env
```

Edit `.env` so `SECRET_API_KEY` holds a value of your own choosing. Run `git status` and confirm that `.env` does not appear: the `.gitignore` entry from the previous exercise is doing its job.

COMMIT THE EXAMPLE ONLY:

```
git add .env.example .gitignore
git commit -m "Add .env.example and update .gitignore"
```

The contrast is immediate. The example is now public, the real values stay on the machine.

THE NAMING CONVENTION is upper-case identifiers with underscores. Environment variables are case sensitive on Linux but case insensitive on parts of Windows; convention picks the form that works everywhere.

IF `.env` APPEARS IN `git status`, the `.gitignore` entry is missing. Add it now and verify again before committing anything. A secret committed once is in the history forever, even if the next commit deletes it.

Exercise: Scaffolding the Project

SCAFFOLD ONLY WHAT WE NEED RIGHT NOW. Other folders appear in the block that first needs them; the rest of the project map you saw in Block 1 stays intentionally empty until then.

```
mkdir -p app data models
touch app/__init__.py
```

CONFIRM THE SHAPE:

```
ls -la
```

The repository now contains `app/`, `data/`, `models/`, `requirements.txt`, `.env.example`, `.env`, `setup-server.sh`, the existing `README.md`, and `.gitignore`. The skeleton matches the map from Block 1 for the parts this block requires.

COMMIT THE SCAFFOLD:

```
git add app/
git commit -m "Add project scaffold"
git push
```

EMPTY FOLDERS MATTER. Git does not track empty directories, only files. `app/__init__.py` exists both to mark the folder as a Python package and to give Git a file to commit so the directory survives `git clone`.

WHY THREE FOLDERS, NOT SEVEN? The map you saw in Block 1 lists ten directories. Creating them all now would be a lie about the state of the project: empty folders that will not be touched for weeks confuse the reader and clutter `ls`. Each folder appears in the block that first uses it.

Exercise: Replicate on the Server

THE REPRODUCIBILITY PROMISE is verifiable only by running the recipe on a different machine. SSH into the prod VM, pull the latest code, and recreate the environment from the two files you committed:

```
ssh produser@192.168.56.11
cd /opt/pixelwise
git pull
bash setup-server.sh
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

VERIFY THE AUDIT ON prod:

```
pip install pip-audit
pip-audit
```

The same advisories should appear as on dev. Identical inputs, identical outputs. This is what reproducibility looks like in practice.

A NOTE ON WHAT COMES NEXT. The repository now has a scaffold, pinned dependencies, and a provisioned server, but no application code. `app/` is an empty Python package waiting for content. In the next block we drop a trained model into `models/`, write the inference layer in `app/classifier.py`, and verify end to end that PixelWise can classify a digit before any API or frontend exists.

WATCH THE INSTALL OUTPUT. `pip` resolves the dependency graph from the lock-shaped `requirements.txt` and downloads each package at the pinned version. The output you see on prod should match the output you saw on dev the first time you ran `pip install`; if it does not, something has drifted.

FROM THIS BLOCK ONWARDS, the GitHub repository is the single source of truth. prod only ever pulls; it never authors. A `git pull` on prod should always be the deployment step, never a place to make edits.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thoughts during and after the exercises:

- Why do we use a virtual environment instead of installing packages system-wide?
- What is the difference between `.env` and `.env.example`, and why do we need both?
- What does `pip freeze` capture that you did not explicitly install, and why does it matter?
- If a colleague clones your repository, which two commands do they run to get a working environment?
- What is the risk of committing `.env` to Git, even briefly, and what would the recovery look like?
- Why does the course teach virtual environments before containers, even though many production systems use containers?

RECAP of key concepts:

- `setup-server.sh` captures system-level provisioning so any fresh VM can be brought up with a single command.
- Virtual environments isolate Python packages per project; never install globally.
- `requirements.txt` pins every dependency so both machines run identical code.
- `.env.example` documents the configuration contract; `.env` holds the real values and never enters Git.
- `pip-audit` catches known vulnerabilities in the dependency graph and runs anywhere a Python environment lives.

MILESTONE. PixelWise has `app/`, `data/`, `models/`, a virtual environment, pinned dependencies, a setup script, and a minimal `.env.example`. The `.gitignore` is doing real work. Anyone who clones the repository can recreate the environment in a handful of commands. The next chapter integrates a trained machine learning model so the project can actually classify a digit.

WITHOUT WRITING APPLICATION CODE YET, we already have a reproducible system on both machines. Anyone with the repository and the script can reach the same starting line.

TEASER. What does it take to turn a trained model file into something the rest of the application can call?

Integrate the Model

YOUR ML COURSE TEACHES YOU TO TRAIN MODELS. THIS BLOCK TEACHES YOU TO SHIP ONE.

The Why

Block 3 left us with a reproducible environment on both machines. The virtual environment is ready, the dependencies are pinned, the project structure is in place. What is missing is the part of the system that does any actual work: PixelWise still cannot classify anything, because the `app/` folder is empty.

ACROSS TEN LECTURE BLOCKS WE BUILD PIXELWISE. This block is the first that produces application logic. Everything before it was foundation: a place to put code, a way to track it, a reproducible environment to run it in. Everything after it depends on the inference interface produced here.

THE GAP BETWEEN A TRAINED MODEL AND A WORKING SYSTEM is larger than most students expect. A `.pkl` file sitting in a folder is not a product. It needs to be loaded safely, called with the exact input format it was trained on, and wrapped in an interface the rest of the application can depend on. Any of these can be wrong without producing an error: the model returns a prediction, the prediction is plausible, the prediction is wrong.

THIS BLOCK BRIDGES THAT GAP. We take a pre-trained digit classifier, integrate it into the PixelWise codebase, and verify end to end that it works before any API or frontend code exists. The pipeline you build here, model file in, prediction dictionary out, is the contract every later block consumes without modification.

Hands On Experience

PIXELWISE now has a scaffold, pinned dependencies, a provisioned server, and a `.env.example` that documents its configuration contract. But the `app/` directory is empty. Before we can build an API or a frontend, we need the core capability: a trained model that can classify a handwritten digit.

SILENT FAILURE is the signature bug of machine-learning systems. Code that crashes is recoverable; you read the trace and fix the line. Code that returns a wrong answer with the same confidence as a right one goes unnoticed until somebody notices the impact downstream. Most of the engineering in this block exists to turn silent failures into loud ones at the earliest possible moment.

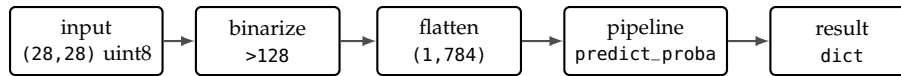


Figure 5: The inference pipeline for one image. The same pipeline scales to a batch by stacking inputs into an $(N, 28, 28)$ array; `predict_proba` is vectorised, so the cost per image stays nearly constant as N grows.

CONSIDER THIS SCENARIO: you download a model file from a colleague, drop it into your project, and write a quick script that calls `model.predict(image)`. It runs without errors. The predictions look reasonable. You ship it. Two weeks later a user reports that the app never predicts the digit 0. You check: the model was trained on digits 1 to 9 only. Nobody documented the omission. Nobody checked.

THE MODEL CONTRACT is the agreement between the training script that produced the file and the inference code that loads it. The contract has three parts: which classes the model predicts, what shape and dtype the inputs must take, and what preprocessing has already been baked in. Honouring the contract is the difference between a service that classifies digits and a service that returns plausible nonsense.

THIS FOURTH BLOCK integrates a trained model into PixelWise and builds the inference layer. By the end you will have

- understood what a model file is and why it is not source code,
- loaded the model safely and verified its class contract at startup,
- implemented a batch-first inference interface in `app/classifier.py`,
- written a smoke test that proves end-to-end correctness on real samples,
- and documented the model's capabilities and limitations in a model card.

A SILENT FAILURE is one the program never reports. Unlike a crash, it has no log line, no stack trace, no obvious moment of failure. The only way to find it is to compare the output against an expected value, and the only way to do that systematically is to write the expectation down somewhere the program can check.

THE INFERENCE INTERFACE you build today is consumed, unchanged, by Block 5's API, Block 6's database writer, Block 7's frontend, Block 8's CI pipeline, and Block 9's analytics. Five later blocks read from the function you write this afternoon. Designing it well is leverage.

What a Model File Actually Is

A `.pkl` FILE IS A BUILD ARTEFACT, not source code. You distribute it; you do not commit it. Just as a compiled binary is produced by a compiler, a model file is produced by a training script. The training script is the source of truth: it takes data, runs an algorithm, and writes weights, hyperparameters, and any preprocessing into a single serialised object.

INSPECT THE ARTEFACT in a Python shell to see what is inside:

```
import joblib
model = joblib.load("models/digit_classifier_v1.pkl")
print(type(model))
# <class 'sklearn.pipeline.Pipeline'>
print(model.classes_)
# ['1' '2' '3' '4' '5' '6' '7' '8' '9']
```

The object that comes back is a full sklearn Pipeline, not just a weights tensor. It carries the preprocessing, the classifier, and the class labels in one package, so loading the file is enough to start predicting.

LOADING A `.pkl` FROM AN UNTRUSTED SOURCE CAN EXECUTE ARBITRARY CODE. pickle and joblib deserialise objects by reconstructing them, including any code embedded in the file. This is convenient when you produced the file yourself; it is dangerous when the file came from somewhere you do not control. Treat `.pkl` files the way you treat executables: only run what you trust.

THE COURSE PROVIDES `models/digit_classifier_v1.pkl`, a LogisticRegression pipeline trained on MNIST digits 1 to 9 (9 classes, ~54,000 training samples, public domain). Class 0 is intentionally held back; students can add it later without collecting any new data.

THE TRAINING SCRIPT `train.py` is provided as supplementary material. Students who want to understand how the artefact was produced can run it: MNIST downloads automatically via `sklearn.datasets.fetch_openml`, and training takes ~30 seconds on CPU. Further reading on training practices: Karpathy, A Recipe for Training Neural Networks: <https://karpathy.github.io/2019/04/25/recipe/> Google, Rules of Machine Learning: <https://developers.google.com/machine-learning/guides/rules-of-ml>

SAFER FORMATS EXIST. ONNX stores only weights and graph structure, no executable code. SafeTensors does the same for PyTorch tensors. For this course joblib is sufficient because we control the training pipeline end to end; for any model received from outside, prefer the safer formats. ONNX: <https://onnx.ai/> SafeTensors: <https://huggingface.co/docs/safetensors>

Honouring the Model Contract

EVERY MODEL WAS TRAINED ON INPUTS IN A SPECIFIC FORMAT. The format is part of the contract: a 28-by-28 greyscale image, pixel values in $[0, 1]$, flattened to 784 features. Violate the format and the model predicts silently wrong: no error, no warning, just a different number than the right one. This bug class is called train/serve skew and it is the most common production ML defect.

THE MNIST MODEL EXPECTS:

- a 28×28 greyscale image,
- pixel values normalised to $[0, 1]$,
- flattened to a 784-element vector.

THE CANVAS IN THE BROWSER PRODUCES UINT8 VALUES in the range $[0, 255]$. Converting to binary (0 or 1) reduces payload size, removes anti-aliasing noise, and matches the training-time preprocessing exactly:

```
import numpy as np
raw = np.array(...)           # (28, 28) uint8
binary = (raw > 128).astype(float) # 0s and 1s
flat = binary.reshape(1, -1)  # (1, 784)
```

THE SKLEARN PIPELINE bundles preprocessing and model into one serialised object. Loading the pipeline is loading the preprocessing. There is no separate normalisation step that a future maintainer might forget; the same object that predicts also normalises, in the order the training script established.

WHY SILENT? The model takes a vector of floats and returns a class index. At no point does it know whether the floats came from the training distribution or from some unrelated source. A photograph, a random scribble, or a correctly drawn digit with the wrong dynamic range all produce the same kind of output: a class label and a confidence score. The label might be wrong, but the system has no way to tell.

BINARIZATION is aggressive but effective for MNIST. Anti-aliased grey edges become sharp black and white. The model was trained on binarized inputs, so serving raw greyscale would introduce train/serve skew immediately, with no exception to mark the moment it happened.

scikit-learn: <https://scikit-learn.org/>
 joblib: <https://joblib.readthedocs.io/>
 NumPy: <https://numpy.org/>

Designing the Inference Interface

BEFORE WRITING ANY CODE, design the contract. Start with the class roster, then the function signatures, then the loading strategy. Each is a decision that propagates: every later block reads from this interface, and every change to it ripples through the API, the database schema, and the frontend.

A NAMED CONSTANT makes the valid output set explicit and independent of the model file:

```
CLASSES = ["1", "2", "3", "4", "5", "6", "7", "8", "9"]
# must match model.classes_, verified at startup
# note: "0" is intentionally absent in v1
```

This list belongs in the code, not only in the model. The frontend, the database schema, and the API documentation all reference it. If the model is swapped for one trained on different classes, the assertion described below fires loudly at startup rather than returning silently wrong labels for the rest of the service's lifetime.

THE FUNCTION SIGNATURES fall out of the contract:

```
def classify_batch(images: np.ndarray) -> list[dict]:
    """
    Args:
        images: (N, 28, 28) uint8, values 0-255
    Returns:
        List of N dicts:
        {"prediction": str, "confidence": float,
         "scores": dict[str, float]}
    """
    if images.ndim != 3 or images.shape[1:] != (28, 28):
        raise ValueError(
            f"Expected (N,28,28), got {images.shape}")
    arr = (images > 128).astype(float).reshape(
        len(images), -1)
    probs = _pipeline.predict_proba(arr)
    return [
        {"prediction": CLASSES[p.argmax()],
         "confidence": float(p.max()),
         "scores": dict(zip(CLASSES, p.tolist()))}
        for p in probs
    ]
```

BATCH-FIRST DESIGN. What happens when 50 students submit a drawing in the same second? If the server processes each request individually, the model is called 50 times with a (1,784) array. If it batches them, the model is called once with a (50,784) array. sklearn's `predict_proba` is natively vectorised, so the cost is nearly identical. Design around batch; single-user is a special case.

```
def classify(image: np.ndarray) -> dict:
    """Single-image convenience wrapper."""
    return classify_batch(image[np.newaxis])[0]
```

LOAD `_pipeline` ONCE AT MODULE LEVEL:

```
import joblib, os
_pipeline = joblib.load(os.getenv("MODEL_PATH"))
assert list(_pipeline.classes_) == CLASSES, \
    f"Model/CLASSES mismatch: {_pipeline.classes_}"
```

Deserialising on every request kills throughput; the model stays in memory for the lifetime of the process. The assertion is the key teaching moment: it fires at startup if the model's classes do not match the code constant. A swapped model with the wrong classes is caught the moment the service starts, not the next time a user gets a wrong digit.

THE WRAPPER IS FOR TESTS ONLY.

The API in Block 5 always calls `classify_batch`, even for a single drawing: it passes a (1,28,28) array. When a request queue is added later to amortise inference cost, no signature changes are needed; the queue assembles a batch and the function consumes it.

`MODEL_PATH` FROM `.env`. The path is configuration, not code. Swapping the model in production becomes an env-var change plus a service restart, not a code change and a deploy. This is the same pattern as the database URL in Block 6 and the API key from Block 3.

The Model Card

A MODEL CARD is the documentation artefact that links a model to its provenance and limitations. It answers four questions any consumer of the model should know the answer to before they trust it: where did the data come from, what can the model do, what does it fail at, and what is it intended for.

FOR PIXELWISE the model card lives next to the artefact in `models/MODELCARD.md` and is committed to Git, so it travels with the repository even though the `.pkl` itself does not:

```
# MODELCARD.md: digit_classifier_v1

## Training Data
MNIST digits 1-9, ~54k train / ~9k test, public domain.
Class 0 withheld intentionally.

## Capabilities
Predict handwritten digits 1-9.
Expected accuracy: ~92% (LogisticRegression baseline).

## Known Failures
6/9 confusion, 3/8 confusion, messy or rotated digits.

## Intended Use
28x28 canvas drawings.
Out of scope: photos, non-digit characters.
```

WHAT THE CARD IS NOT is a substitute for the runtime checks already built into the loader. Documentation tells humans what to expect; the assertion at module load tells the program what to expect. Both are needed. Documentation drifts; assertions cannot.

Model cards as a concept were proposed by Mitchell et al. in 2018 and are now standard at Google, HuggingFace, and increasingly across the industry. Model Cards (Google): <https://modelcards.withgoogle.com/> HuggingFace Model Cards: <https://huggingface.co/docs/hub/model-cards>

THE CARD IS A CONTRACT, like any other. When v2 is deployed via the Block 8 feature flag, the card is updated alongside it; the version number, the training data section, and the capabilities all change in lockstep. Traceability across versions is the entire reason the card is a separate file.

Exercise: Inspecting the Model Artefact

CATCHING UP VIA THE REFERENCE TAG. The course maintains a public reference repository at <https://github.com/schutera/pixelwise> with a tag at the end of every block. v0.2 marks the end of Block 3; v0.3 will mark the end of this block. If you joined late or your repository has drifted, reset to the previous tag before starting:

```
git fetch origin
git reset --hard v0.2
```

This rewrites your working tree to match the end of Block 3 exactly. The state on disk is now identical to the published reference, including `setup-server.sh`, `requirements.txt`, and `.env.example`.

BLOCK 2 INTRODUCED THE MODEL ARTEFACT as something you load, not something you commit. Copy `digit_classifier_v1.pkl` (provided) into `models/`. Confirm that `.gitignore` excludes `*.pkl` so the artefact stays out of version control:

```
ls models/
git status
```

INSPECT THE ARTEFACT in a Python shell:

```
source .venv/bin/activate
python3
>>> import joblib
>>> model = joblib.load("models/digit_classifier_v1.pkl")
>>> print(type(model))
>>> print(model.classes_)
```

What type is it? What classes does it predict? Does class 0 appear? The answer to the third question determines what CLASSES should contain in the next exercise.

EXERCISES mirror the four theory sections in order. Read a concept, then practise it; the structure of this block matches the structure of Block 2 one to one. If your repository has drifted from the end of Block 3, the catch-up below brings it back to the known good state.

IF THE FILE APPEARS IN `git status`, your `.gitignore` from Block 3 needs an update. Add `models/*.pkl` on its own line and rerun `git status`. A model file accidentally committed once is in the history forever, even if the next commit deletes it.

Exercise: Preprocessing and Binarization

BLOCK 2 INTRODUCED BINARIZATION as the preprocessing step that matches the training contract. Load a sample MNIST digit and compare the raw and binarized representations side by side:

```
from sklearn.datasets import fetch_openml
import numpy as np

X, y = fetch_openml("mnist_784", version=1,
                    return_X_y=True, as_frame=False)
sample = X[0].reshape(28, 28).astype(np.uint8)
binary = (sample > 128).astype(float)
print(sample[:5, :5])    # raw values
print(binary[:5, :5])    # 0s and 1s
```

PLOT IF YOU HAVE A NOTEBOOK OPEN. A side-by-side image of the raw and binarized digit makes the effect visible at a glance. `matplotlib` is not in `requirements.txt` yet; install it locally with `pip install matplotlib` for this exercise only, then leave it out of the pinned requirements until a later block needs it.

NOTICE THE STRUCTURE SURVIVES.

The raw image has anti-aliased grey edges; the binarized image is sharp black and white. The digit is still recognisable, the noise is gone, and the input now matches what the model saw during training. Serving raw greyscale would mean predicting against a distribution the model has never seen.

WHY THRESHOLD AT 128? 128 is the midpoint of the `uint8` range, the simplest split between “ink” and “paper”. Otsu’s method picks a smarter threshold per image, and for noisier inputs that is sometimes worth doing. For MNIST the simple split is enough and matches the training pipeline exactly.

Exercise: The Classifier Module

BLOCK 2 INTRODUCED THE INFERENCE INTERFACE. Implement `app/classifier.py` with the class constant, module-level loading, the startup assertion, and the two functions:

```
import os
import joblib
import numpy as np
from dotenv import load_dotenv

load_dotenv()

CLASSES = ["1", "2", "3", "4", "5", "6", "7", "8", "9"]

_pipeline = joblib.load(os.getenv("MODEL_PATH"))
assert list(_pipeline.classes_) == CLASSES, \
    f"Model/CLASSES mismatch: {_pipeline.classes_}"

def classify_batch(images: np.ndarray) -> list[dict]:
    if images.ndim != 3 or images.shape[1:] != (28, 28):
        raise ValueError(
            f"Expected (N,28,28), got {images.shape}")
    arr = (images > 128).astype(float).reshape(
        len(images), -1)
    probs = _pipeline.predict_proba(arr)
    return [
        {"prediction": CLASSES[p.argmax()],
         "confidence": float(p.max()),
         "scores": dict(zip(CLASSES, p.tolist()))}
        for p in probs
    ]

def classify(image: np.ndarray) -> dict:
    return classify_batch(image[np.newaxis])[0]
```

UPDATE `.env` AND `.env.example` so the loader knows where to find the artefact:

```
MODEL_PATH=models/digit_classifier_v1.pkl
```

WRITE `predict.py` AS A SMOKE TEST. Load five MNIST test samples, call `classify_batch`, print the predictions next to the ground truth:

THE ASSERTION IS THE HEADLINE. Comment it out, save, restart Python, and import the module. Now mutate `CLASSES` to add a fake "10" entry, restore the assertion, restart, and observe the assertion firing. The startup error message is exactly what you want to see when a model swap goes wrong.

```

from app.classifier import classify_batch
from sklearn.datasets import fetch_openml
import numpy as np

X, y = fetch_openml("mnist_784", version=1,
                    return_X_y=True, as_frame=False)
images = X[:5].reshape(-1, 28, 28).astype(np.uint8)
truth = y[:5]
results = classify_batch(images)
for r, t in zip(results, truth):
    print(f"Pred: {r['prediction']} "
          f"(conf {r['confidence']:.2f}) True: {t}")

```

READ `predict.py` AS THE API
 PREVIEW. This is what the API in
 Block 5 will do: take pixel data, call
`classify_batch`, return the result.
 Block 5 swaps `stdin` for HTTP and
`stdout` for JSON; the inference logic is
 unchanged.

Exercise: The Model Card

BLOCK 2 INTRODUCED THE MODEL CARD as the documentation artefact that travels with the project even though the `.pkl` does not. Write `models/MODELCARD.md` following the structure from the theory section: dataset, input format, performance per class, known failures, version, and date.

COMMIT THE NEW FILES, but not the `.pkl`:

```
git add app/classifier.py predict.py \
      models/MODELCARD.md .env.example
git commit -m "Add classifier module and model card"
git push
```

The repository is now at the state `v0.3` marks: a working classifier, a smoke test, a model card, and an updated configuration contract. The `.pkl` stays on each developer's machine, the artefact path is in `.env`, and the rest of the team can reproduce the environment from the code in Git.

USE THE VALUES YOU MEASURED. Replace "`~92%`" with whatever `predict.py` reports as you expand it across the test set. The numbers you cite should match the numbers your code produces; a model card with optimistic round figures is the start of the documentation drift the previous section taught us to avoid.

VERIFY THE SMOKE TEST ON prod. SSH into the server, pull, copy the `.pkl` into `/opt/pixelwise/models/`, set `MODEL_PATH` in the production `.env`, and run `python predict.py`. Identical output on both machines is the deliverable of this block.

Optional: Add the Missing o

THE DIGIT CLASS "0" WAS INTENTIONALLY EXCLUDED FROM v1. To produce v2: add "0" to CLASSES, include MNIST's class-0 samples in the training split, re-run `train.py`, save as `digit_classifier_v2.pkl`. The assertion will break the moment v2 is loaded against v1's CLASSES; fix it by updating the constant. Update `MODEL_CARD.md` to document that v2 covers all ten digits and note any accuracy change.

COULD WE HAVE SPOTTED THE MISSING CLASS AT RUNTIME?

- After a few hundred predictions, a missing class is invisible: the model never predicts it, users never see an error, the database just never accumulates any os.
- When the model sees an input it was not trained on, it is often less certain. A systematic drop in average confidence could indicate out-of-distribution inputs: a weak signal, but a runtime signal that requires no labelled data.
- Returning `null` when `confidence < threshold` turns low confidence into a monitoring signal: the volume of rejected predictions is itself a metric.

A NOTE ON WHAT COMES NEXT. `app/classifier.py` works, but it is trapped in a Python script. A user with a browser cannot call it; a teammate on a different machine cannot reach it. The next block wraps `classify_batch` in an HTTP endpoint, defines the request and response contract with Pydantic, and runs the result as a managed systemd service so the model becomes reachable without a human in the loop.

This challenge connects directly to Block 8: v2 is deployed via the feature flag, and Block 9's analytics compare v1 and v2 performance. No data collection is needed; MNIST has the os already.

SYSTEMATIC DETECTION OF SILENT FAILURES is the domain of MLOps: uncertainty estimation, distribution shift, missing classes, silent degradation. This surface is intentionally shallow here. A dedicated MLOps course covers it in depth.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thoughts during and after the exercises:

- What is train/serve skew, and why is it the most common production ML defect?
- Why do we load the model once at module level instead of on every request?
- What does the startup assertion protect against, and what would happen without it?
- Why is the `.pkl` in `.gitignore` but `MODEL_CARD.md` is committed?
- How would you detect at runtime that the model is missing a class it should predict?
- Why is batch-first design the right default, even when most requests carry a single image?

RECAP of key concepts:

- A `.pkl` file is a build artefact, not source code; never commit it to Git.
- Train/serve skew is silent by default; the startup assertion is the loudest defence.
- Batch-first design scales from one user to N concurrent users with no code changes.
- Module-level loading keeps the model in memory for the lifetime of the process.
- The model card documents capabilities, limitations, and provenance, and updates with the model.
- `predict.py` proves end-to-end correctness before any API exists.

MILESTONE. `app/classifier.py` exposes a batch-first inference interface that scales from one student to N concurrent users with no code changes. `predict.py` proves it works end to end on real MNIST samples. `MODEL_CARD.md` documents what the model is and is not. The next chapter wraps the inference function in a FastAPI service so the model becomes reachable over HTTP.

SECURITY THREAD. Never load a `.pkl` from an untrusted source; `joblib` and `pickle` deserialisation execute arbitrary code. `MODEL_PATH` comes from `.env`, never hard-coded, so swapping the model in production requires only an env-var change, not a code change. Training data and model artefacts are not committed to Git.

TEASER. The classifier is callable from Python, but who, outside your terminal, can reach it?

Index

- [.env](#), 35, 40
- [.env.example](#), 40
- [.gitignore](#), 29
- [.pkl](#), 51, 53
- [/etc](#), 19
- [/opt](#), 19
- [12-factor app](#), 34, 40

- [abstraction](#), 42
- [Ansible](#), 37
- [apt](#), 35
- [assertion](#), 60
- [assessment](#), 9

- [Bash](#), 14, 37
- [batch inference](#), 55
- [binarization](#), 54, 59

- [chmod](#), 14
- [chown](#), 14
- [classifier](#), 50
- [conda](#), 38
- [containers](#), 42
- [continuous integration](#), 28
- [course work](#), 9
- [CVE](#), 39, 46

- [data hygiene](#), 25
- [dependencies](#), 34
- [dev VM](#), 11
- [distribution](#), 14
- [Docker](#), 42
- [Dockerfile](#), 42
- [DVC](#), 25

- [Ed25519](#), 15
- [environment variables](#), 40
- [exercises](#), 17, 19, 20, 30–33, 43, 45–49, 58–60, 62

- [fork](#), 28

- [Git](#), 21
 - [-u flag](#), 28
 - [blob](#), 25
 - [branch](#), 25
 - [branching](#), 26
 - [clone](#), 28
 - [commit](#), 25
 - [data model](#), 25
 - [default branch](#), 32
 - [diff](#), 45
 - [empty directory](#), 48
 - [fetch](#), 28
 - [fork](#), 28
 - [HEAD](#), 25
 - [history](#), 29
 - [HTTPS clone](#), 33
 - [install](#), 30
 - [LFS](#), 25
 - [merge conflict](#), 27
 - [merging](#), 26
 - [pull](#), 28
 - [push](#), 28, 33
 - [remotes](#), 27
 - [repository](#), 26
 - [staging](#), 26
 - [staging area](#), 26
 - [status](#), 32
 - [tag](#), 43, 58
 - [tags](#), 27
 - [tree](#), 25
 - [working directory](#), 26
- [GitHub](#), 21, 27
 - [username](#), 33

- [Hugging Face](#), 25
- [hypervisor](#), 13

- [idempotence](#), 37
- [inference](#), 50
- [inference interface](#), 55
- [interface contracts](#), 24
- [isolation](#), 38
- [issues](#), 28

- [joblib](#), 50, 53, 54

- [KeePass](#), 15

- [least privilege](#), 14
- [license](#), 2
- [Linux](#), 9, 14
- [lock file](#), 41
- [LogisticRegression](#), 53

- [MLOps](#), 63
- [MNIST](#), 11, 50
- [model artefact](#), 51
- [model card](#), 57, 62
- [model contract](#), 54
- [model integration](#), 50
- [model versioning](#), 63
- [MODEL_PATH](#), 56
- [MODELCARD.md](#), 57

- [nano](#), 16, 32
- [NumPy](#), 54

- [ONNX](#), 53
- [OpenSSH](#), 15
- [Oracle VirtualBox Manager](#), 9
- [os.getenv](#), 40

- [pickle](#), 53
- [pip](#), 34, 39
- [pip audit](#), 39, 46
- [PixelWise](#), 11, 35, 51

poetry, 41
predict.py, 61
project report, 9
project structure, 34
Projektbericht, 9
provisioning, 37, 43, 62
pull request, 28
PuTTY, 15
PyPI, 39
pyproject.toml, 41
python-dotenv, 40

recap, 21, 50, 64
recovery, 43
reflection, 21, 50, 64
Remote-SSH, 15
reproducibility, 35
requirements.txt, 35, 39, 46

scaffolding, 48
scikit-learn, 54
secrets, 29

security, 64
semantic versioning, 27, 39
server, 11
set -euo pipefail, 37
setup-server.sh, 35, 37
shebang, 37
shell prompt, 45
silent failure, 51
sklearn
 Pipeline, 53
snapshot, 13
SSH, 9, 15
 config, 30
 key, 30
 keygen, 30
 login, 33
 private key, 30
 public key, 30
ssh-agent, 15
sudo, 30
systemctl, 16
systemd, 16

threshold, 59
tig, 27
touch typing, 19
train.py, 53
train/serve skew, 51, 54
transitive dependencies, 41
type-1 hypervisor, 13
type-2 hypervisor, 13

Ubuntu, 14
 terminal, 30
Unix philosophy, 14
uv, 41

venv, 38
version control, 21, 23
Virtual Machines, 9, 13
virtualenv, 34, 38
VS Code, 15

Weights & Biases, 25
whoami, 14